Concurrency Control & Distributed Database Design

A mini banking system · Chimaroke Amadi · GoMyCode MSC checkpoint · 7 November 2026

> I didn't want to answer only on paper, so I ran every scenario in this report on a real **MySQL 9.7.4** (InnoDB) server. Two connections played the two users, and I forced the operations into the exact order shown in each table. All balances and timelines below are the real results, generated by `demo/concurrency.js` and `demo/distribution.js`.

Part 1: Transaction Management

The scenario

The `accounts` table starts with three accounts: **A** (Amira) = 1,000 TND, **B** (Bilel) = 500 TND and **C** (Cyrine) = 300 TND. Two transfers start at the same time, and both take money from **the same account A**:

- **T1**: transfer 200 from A to B
- **T2**: transfer 300 from A to C

Whatever order they run in, the correct result is A = **500**, B = 700, C = 600. The bank's total money must stay at **1800**, because a transfer only moves money and never creates it.

1. The concurrency issue: a lost update

A transfer is a *read-modify-write*: read A's balance, subtract the amount in the application, then write the new balance. If both transactions read A **before** either one writes, they both start from 1000. T1 writes 800, then T2 writes 700 and **overwrites** T1's update, as if T1 had never touched account A. This is the **lost update** problem.

Schedule 1: no locking (real run)

#	T1: Amira → Bilel (200)	T2: Amira → Cyrine (300)	A in DB	Note
1	read(A) = 1000		1000	
2		read(A) = 1000	1000	T2 reads the same old value
3	write(A = 800), B += 200		1000	
4	COMMIT		800	
5		write(A = 700), C += 300	800	overwrites T1's write: LOST UPDATE
6		COMMIT	700	
Final: A = **700**, B = 700, C = 600 · total money = **2000** (must stay 1800) · Schedule safe? NOT SAFE				

> **Result:** A ended on **700** instead of 500, but B and C still got their money. The bank's total went from 1800 to **2000**: **200 TND appeared out of nowhere**. MySQL raised no error. Its default isolation level (`REPEATABLE-READ`) does not stop this pattern when the new value is calculated in the application.

Another issue I tested: the dirty read. If a user viewing balances can read *uncommitted* data, they might see money that never really moved. While T1 was in progress, a reader at `READ UNCOMMITTED` saw A = **800**.

Then T1 rolled back, and the real balance was 1000. At `READ COMMITTED` or higher, the reader saw 1000, which was correct. So **"view balance" must never run at READ UNCOMMITTED**.

2. The locking mechanism

There are two kinds of locks:

- **Shared lock (S)** for reading: many transactions can hold one at the same time.
- **Exclusive lock (X)** for writing: only one transaction can hold it, and it blocks both S and X locks from others.

Lock compatibility

held \ requested	S	X
S	✔	✘ wait
X	✘ wait	✘ wait

> **My proposal:**
>
> - **Transfers** take an **exclusive (X) lock** on *both* account rows *before reading* them, with `SELECT … FOR UPDATE`. They keep the locks until `COMMIT` (strict two-phase locking).
> - **Viewing a balance** only needs a **shared (S) lock** (`SELECT … FOR SHARE`), or simply a normal committed read.
> - Accounts are always locked in the **same order (smallest id first)**. Then two opposite transfers (A→B and B→A) can't deadlock each other.

```
START TRANSACTION;
-- lock both rows, smallest id first
SELECT id, balance FROM accounts WHERE id IN (:from, :to) ORDER BY id FOR UPDATE;
-- (in the app: check that balance(:from) >= :amount, otherwise ROLLBACK)
UPDATE accounts SET balance = balance - :amount WHERE id = :from;
UPDATE accounts SET balance = balance + :amount WHERE id = :to;
COMMIT;   -- releases the locks
```

Why not shared locks alone? I tried it. Both transactions got an S-lock on A, because S-locks are compatible. Then each one needed to upgrade to an X-lock to write, and each was waiting for the other. MySQL detected the **deadlock** (error 1213 `ER_LOCK_DEADLOCK`) and rolled T2 back. The result was correct after the retry, but only because MySQL broke the deadlock. For read-modify-write, the lock has to be exclusive **from the start**.

Shared locks only (real run): deadlock, then T2 retries

#	T1: Amira → Bilel (200)	T2: Amira → Cyrine (300)	A in DB	Note
1	S-lock(A), read(A) = 1000		1000	
2		S-lock(A), read(A) = 1000	1000	shared locks are compatible
3	write(A) → needs X-lock, WAITS for T2		1000	
4		write(A) → DEADLOCK, rolled back	1000	MySQL error 1213 ER_LOCK_DEADLOCK
5	lock granted, write(A = 800), B += 200, COMMIT		800	
6		retry: read(A) = 800, write(A = 500), COMMIT	500	
Final: A = **500**, B = 700, C = 600 · total money = **1800** (must stay 1800) · Schedule safe? **NOT SAFE as written: it deadlocked, and only ended correct because MySQL rolled T2 back and the app retried it**				

3. Pessimistic or optimistic?

	Pessimistic (lock first)	Optimistic (check at the end)
How	`SELECT … FOR UPDATE` : others wait	a `version` column: `UPDATE … WHERE version = :readVersion`
On conflict	T2 waited 311 ms, then continued with the new balance	T2's update matched 0 rows → rollback + 1 retry
Final result	A = 500 · ✔ yes	A = 500 · ✔ yes
Best when	conflicts are likely and a failure is expensive	conflicts are rare (e.g. editing a profile)

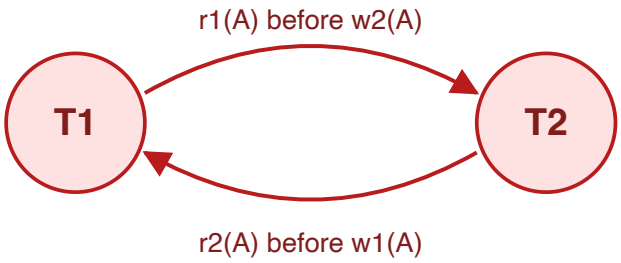
I would use pessimistic locking for transfers, for four reasons:

- **Conflict is part of the scenario.** Two users are hitting the *same account*. With optimistic locking, one of them is guaranteed to fail and redo the work, and under heavy load this could keep happening.
- **It's money.** The cost of getting it wrong is very high. Pessimistic locking makes it impossible for two transfers to work on the same balance at the same time. Optimistic locking only works if *every* piece of code remembers the version check.
- **Transfers are short**: two rows, a few milliseconds. The locks are held very briefly, so waiting is cheap (here it was 311 ms, only because I paused T1 on purpose).
- **The user experience is simpler.** T2 just waits a moment. It doesn't get a "please try again" error.

I'd keep optimistic locking for low-conflict data, like a customer updating their address.

4. Order of operations: is the schedule safe?

Schedule 1 (no locking) is shown above. Written in short form, it is `r1(A) r2(A) w1(A) w2(A)`. It is **not conflict-serializable**: `r1(A)` comes before `w2(A)`, which gives an edge T1 → T2, and `r2(A)` comes before `w1(A)`, which gives an edge T2 → T1. The precedence graph has a **cycle**, so no serial order gives the same result. **Not safe.**



Schedule 2 uses my exclusive locks. T2's lock request **waits**, so it reads A only after T1 commits. The schedule becomes equivalent to the serial order **T1 → T2**. **Safe.**

Schedule 2: exclusive locks, SELECT … FOR UPDATE (real run)

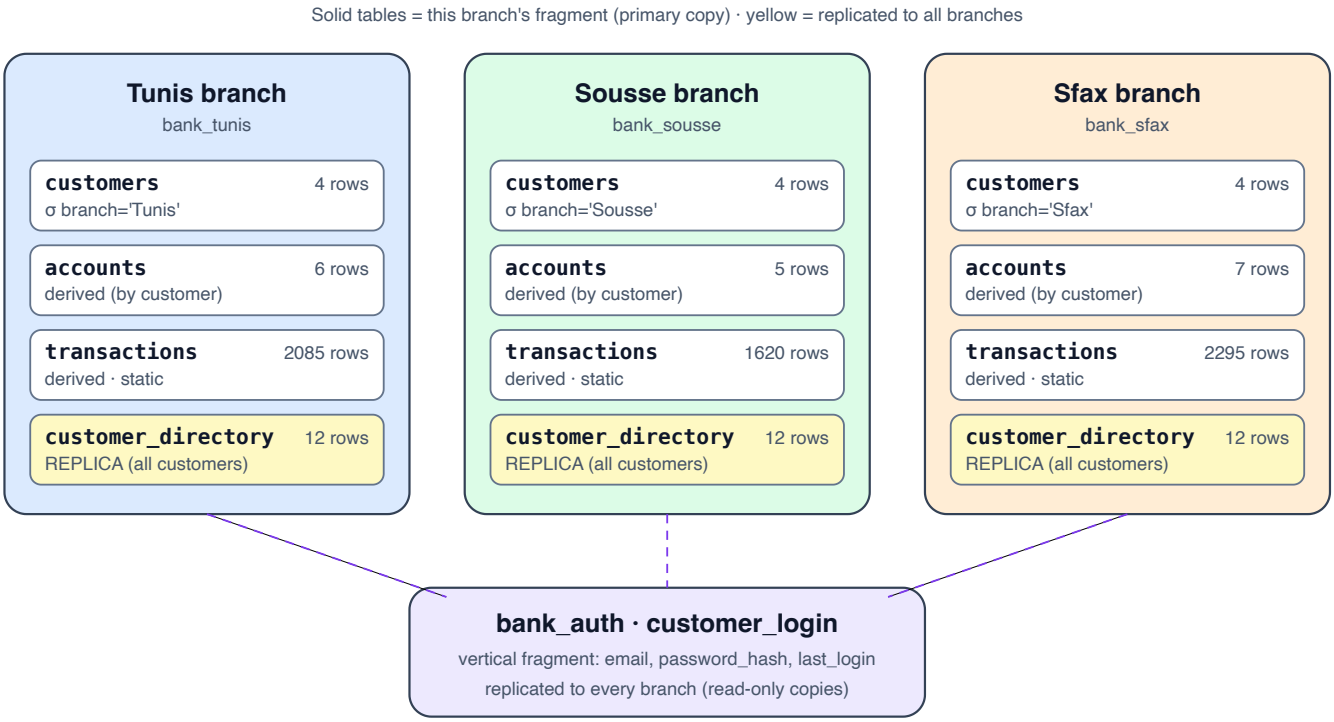
#	T1: Amira → Bilel (200)	T2: Amira → Cyrine (300)	A in DB	Note
1	X-lock(A, B), read(A) = 1000		1000	
2		X-lock(A) requested → WAITS	1000	blocked by T1
3	write(A = 800), B += 200		1000	
4	COMMIT (releases locks)		800	
5		lock granted, read(A) = 800	800	T2 sees the NEW value
6		write(A = 500), C += 300	800	
7		COMMIT	500	
Final: A = **500**, B = 700, C = 600 · total money = **1800** (must stay 1800) · Schedule safe? **SAFE**				

Summary of the four strategies I ran

Strategy	Final A	Total money	Correct?	What happened
No locking	700	2000	✘ no	lost update
Shared locks only	500	1800	✔ yes	deadlock, T2 rolled back and retried
Exclusive locks (pessimistic)	500	1800	✔ yes	T2 waited for T1
Version check (optimistic)	500	1800	✔ yes	T2 detected the conflict and retried

Part 2: Distributed Database Planning

The bank has three branches: **Tunis, Sousse and Sfax**. In my demo, each branch is its own MySQL database (a "site"): `bank_tunis`, `bank_sousse` and `bank_sfax`. The login data goes in `bank_auth`. I started from a central database with 12 customers, 18 accounts and 6,000 transactions, and split it.



1. Horizontal fragmentation of Customers by branch

Each customer belongs to a **home branch**, the branch where they opened their account. I split the rows with one predicate per branch. Each fragment is a *selection* of the Customers table:

```
Customers_Tunis  = σ branch = 'Tunis'  (Customers)
Customers_Sousse = σ branch = 'Sousse' (Customers)
Customers_Sfax   = σ branch = 'Sfax'   (Customers)

CREATE TABLE bank_tunis.customers AS
  SELECT customer_id, full_name, branch, phone, address
  FROM bank_central.customers
  WHERE branch = 'Tunis';
```

Each fragment is stored at its own branch, so most work (opening accounts, deposits, local customers asking for their balance) uses **only local data**, with no network trip. I also added `CHECK (branch = 'Tunis')` to the Tunis fragment. When I tried to insert a Sfax customer there, MySQL refused (`ER_CHECK_CONSTRAINT_VIOLATED`).

The fragments in the demo, and the three correctness rules

Tunis (4)	Sousse (4)	Sfax (4)
1 Amira Ben Salah 4 Dhia Mansour 5 Emna Jaziri 10 Karim Zouari	2 Bilel Trabelsi 6 Fares Bouazizi 9 Ines Chaabane 12 Mehdi Saidi	3 Cyrine Gharbi 7 Ghada Kallel 8 Hamza Dridi 11 Lina Haddad
Completeness (every customer is in a fragment): ✔ yes · **Disjointness** (no customer in two fragments): ✔ yes · **Reconstruction** (`UNION ALL` of the 3 = original table): ✔ yes		

Accounts and **transaction history** follow the same split through **derived horizontal fragmentation**: an account is stored with its owner, and a transaction is stored with its account. This keeps a customer's data together at one site.

Site	Customers	Accounts	Transactions
Tunis	4	6	2,085
Sousse	4	5	1,620
Sfax	4	7	2,295
Total	**12**	**18 ✔**	**6,000 ✔**

2. Vertical fragmentation: the login information

> I would move the **login information (`email` , `password_hash` , `last_login`)** into its own table, `customer_login` . It keeps the key `customer_id` so the two parts can be joined back.

```
CREATE TABLE bank_auth.customer_login AS
  SELECT customer_id, email, password_hash, last_login
  FROM bank_central.customers;
```

- **Security:** credentials are the most sensitive data. In a separate table (here, a separate database), they can have stricter permissions and encryption, and branch staff who look up customer profiles never touch them.
- **Different use:** login data is read on *every* login, from anywhere (mobile app, any branch). The profile (address, phone) is read much less often, and mostly at the home branch.
- **Not tied to a branch:** a customer should be able to log in from any city, so this part is treated differently (see replication below).

Branch sites keep: `customer_id, full_name, branch, phone, address` . `bank_auth.customer_login` keeps: `customer_id, email, password_hash, last_login` . Joining the two on `customer_id` gave back the original table exactly: ✔ yes (a *lossless* vertical split).

3. What should be replicated to all branches?

Data	Replicate to all 3 branches?	Why
Customer info (a directory: id, name, home branch; and the login data)	Yes	It's **small**, it's **read all the time** (identifying a customer, logging in), and it **rarely changes**. With a copy everywhere, any branch can serve any customer, and it knows *where* that customer's accounts live. The rare updates go to one primary copy and are sent to the others. A few seconds of delay is fine for a name or a phone number.
Account balances	No	Balances **change with every transfer** and must **always be exact**. If all three branches had a copy, every transfer would have to update three sites at once (with two-phase commit), which is slow. And if a network link went down, two branches could accept withdrawals from different copies, so money could be spent twice. I keep **one primary copy at the home branch** (plus a backup standby for failures, not used for writes).
Transaction history	No (not to every branch)	It's by far the **largest** data set (6,000 rows for 12 customers in my demo) and it **grows forever**. Each customer's history is mostly needed at their home branch. It's append-only, so I'd only copy it **asynchronously to one central backup/audit site**, not to every branch.

Here's how the directory is used in the demo. Amira Ben Salah, a Tunis customer, walks into the **Sfax** branch. The local copy of `customer_directory` (12 rows at every site) finds her straight away and says her home branch is Tunis. The balance query is then sent to `bank_tunis` (7,173.79 TND), the only place the real balance is stored.

4. Static or dynamic allocation for transaction history?

> **Static allocation.** Each transaction is stored at its account's home branch, decided once when the system is designed.

- **The access pattern is predictable.** History is almost always read by the account owner's branch. We know this in advance, so there's nothing to gain from moving data around automatically.
- **The data is append-only and huge.** Old transactions are never updated. Dynamic allocation would keep moving large amounts of rows between cities, using network bandwidth and creating risk for no real benefit.
- **It's simpler and easier to audit.** Everyone (the queries, the auditors, the backup plan) knows exactly where a transaction lives.

Dynamic allocation would only make sense if usage changed a lot, for example if many customers moved between cities. Even then, I'd handle it with an occasional planned move of that customer's fragment rather than automatic, continuous reallocation.

What I learned

The biggest surprise was that the lost update happened **silently**. MySQL gave no error or warning, and the bank simply ended up with 200 TND that shouldn't exist. Transactions alone don't protect you. You have to choose the right locks (here, an exclusive lock taken before reading), or check a version number. I

also learned that shared locks can make things *worse* for read-modify-write, because they lead to deadlocks.

For the distributed part, the main idea is to **keep data where it's used**: fragment customers, accounts and history by branch. Then **replicate only what is small, read often and rarely changed**, which is the customer directory and login data. Balances need one exact copy, and history is too big and too local to copy everywhere.

Generated from real runs on MySQL 9.7.4 (InnoDB, default isolation REPEATABLE-READ), using `demo/concurrency.js` and `demo/distribution.js`. Run `npm run all` to reproduce.